

Coordination Service

1. Overview

A **Coordination Service** is a distributed component used to help multiple nodes coordinate actions such as:

- Leader election
- Membership discovery
- Configuration management
- Distributed locks
- Service discovery
- Metadata storage
- Ordering / consensus

Instead of every node talking to every other node (full mesh), nodes rely on a coordination layer.

This reduces communication complexity and helps maintain consistency.

2. Core Idea

Rather than:

None

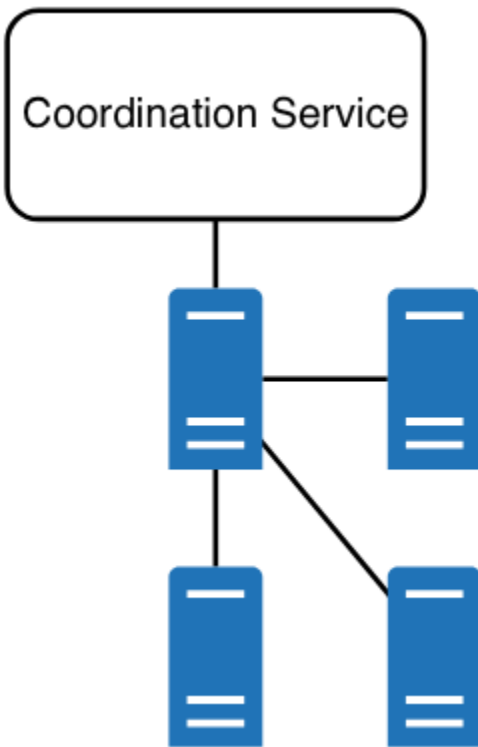
Every node ↔ every node

Use:

None

Nodes ↔ Coordination Service

The coordination service may elect a **leader** or manage shared state.



3. Why It Is Needed

In distributed systems, common questions arise:

None

Who is leader?

Which nodes are alive?

Where is config stored?

Who owns this lock?

Which server should process task?

Without coordination:

- Duplicate leaders
- Conflicts
- Inconsistent configs
- Race conditions
- Hard failover

4. Leader Election

One of the most common uses.

Cluster of 5 nodes:

None

Node A

Node B

Node C

Node D

Node E

Coordination service elects:

None

Leader = Node C

Then:

- Writes go to leader
- Followers replicate
- If leader dies → elect new leader

5. Why Leader Reduces Messages

Without Leader (many-to-many)

Every node may coordinate with all others.

High communication cost.

With Leader

None

Followers → Leader

Leader → Followers

Lower message complexity and clearer ordering.

6. Real-World Examples

Databases

Primary-replica systems:

- MySQL primary/replica
- PostgreSQL leader/followers

Kafka

Controller node coordinates partitions.

Kubernetes

etcd stores cluster state.

HDFS

NameNode coordinates metadata.

7. Popular Coordination Algorithms

A. Paxos

Consensus algorithm ensuring agreement among distributed nodes.

B. Raft

Simpler consensus algorithm, popular in interviews.

C. Zab

Used by Apache ZooKeeper.

8. Popular Coordination Services

Apache ZooKeeper

Classic coordination system.

etcd

Used by Kubernetes.

Consul

Used for service discovery/config.

9. What ZooKeeper Provides

A. Leader Election

Choose exactly one active leader.

B. Membership Tracking

Which nodes are alive?

Uses heartbeats / sessions.

C. Distributed Locks

Only one worker processes critical task.

D. Config Storage

Shared configs centrally stored.

E. Ordered Updates

Clients can observe writes in order.

F. High Performance

Metadata stored in memory with disk persistence.

10. Example: Distributed Lock

Two workers want same job.

Without coordination:

None

Worker A processes

Worker B also processes

Duplicate execution

With coordination service:

None

Worker A acquires lock

Worker B waits

11. Consistency Model

Many coordination systems favor consistency over availability.

Example:

Apache ZooKeeper is commonly considered **CP** under CAP:

- Consistency prioritized
- During partition, some operations may reject

Reason:

Better to reject than allow two leaders.

12. Split Brain (Very Important)

Split brain means two nodes both believe they are leader.

Example:

None

Network partition occurs

Node A thinks leader

Node B thinks leader

Danger:

- Double writes
- Corruption
- Conflicting actions

Coordination systems prevent this using quorum/consensus.

13. Quorum in Coordination

For cluster of N nodes, often need majority.

Example:

None

$N = 5$

Need 3 votes

Ensures only one valid leader.

14. Horizontal Scaling

Coordination clusters usually run odd number nodes:

- 3 nodes
- 5 nodes
- 7 nodes

Too many nodes can slow consensus.

Unlike stateless services, coordination layers scale carefully.

15. Coordination Service vs Config File

Feature	Config File	Coordination Service
Dynamic updates	Poor	Strong
Failure handling	Weak	Strong
Leader election	No	Yes
Locking	No	Yes
Membership	Manual	Automatic

Complexity	Low	High
------------	-----	------

16. Typical Architecture

None

App Nodes



Coordination Cluster

(3 or 5 nodes)



Leader / Config / Locks / Membership

17. Where to Use in HLD

Use coordination service when system needs:

- Active/passive failover
- One leader writes
- Distributed cron jobs
- Shared config
- Node membership
- Strong metadata consistency

18. Where NOT to Use

Avoid for simple apps needing only:

- Static config
- Single server deployment
- No distributed coordination needs

Because it adds operational complexity.

19. Failure Scenarios

Leader Dies

None

Detect timeout

Hold election

Choose new leader

Node Partitioned

May lose leadership if quorum unavailable.

20. Interview Answer

Q: How do multiple nodes agree on one leader?

Answer:

I'd use a coordination service like ZooKeeper or etcd using consensus algorithms such as Raft/Paxos. Nodes register heartbeats, and the system elects exactly one leader using quorum. This prevents split brain and supports failover.

21. Best Practices

- Use odd number nodes (3 or 5)
- Separate metadata from large data storage
- Keep cluster geographically sensible
- Monitor quorum health
- Use timeouts carefully
- Protect access with ACLs
- Backup configs

22. Short Revision Notes

None

Coordination Service manages:

- Leader election
- Membership
- Locks
- Config
- Consensus

Examples:

ZooKeeper, etcd, Consul

Main tradeoff:

Strong guarantees but more complexity